

PROYECTO FINAL — DevOps & Cloud

Materia: DevOps & Cloud

Alumno: Octavio David Noval Rapallo

Comisión: —

Profesor: —

Fecha: Julio 2026

Repositorio: <https://github.com/OctavioDNoval/EntregaFinalCoderHouseDevOps>

Índice

- [1. Introducción](#)
- [2. Contenedores — Docker](#)
- [3. Infraestructura como Código — Terraform](#)
- [4. CI/CD — GitHub Actions](#)
- [5. Orquestación — Kubernetes](#)
- [6. Monitoreo — Prometheus y Grafana](#)
- [7. FinOps — Gestión de Costos en AWS](#)
- [8. Estructura del Proyecto](#)
- [9. Conclusión](#)

1. Introducción

Este proyecto integrador consiste en una **API REST de gestión de tareas (TO-DO)** desarrollada con **Spring Boot 3.4.4 + Java 17 + SQLite**, contenerizada con Docker, orquestada con Kubernetes, desplegada en AWS mediante Terraform, integrada con un pipeline CI/CD en GitHub Actions, y monitoreada con Prometheus y Grafana.

El objetivo es demostrar la aplicación práctica de las principales herramientas y prácticas del ecosistema DevOps, incluyendo contenerización, infraestructura como código, integración y despliegue continuos, orquestación de contenedores, observabilidad y gestión de costos en la nube.

Stack tecnológico

Componente	Tecnología
Aplicación	Spring Boot 3.4.4 + Java 17 + SQLite + JPA
Contenedores	Docker (multi-stage) + Docker Compose
Orquestación	Kubernetes (Deployment, Service, HPA, Ingress)
Infraestructura	Terraform (AWS EC2, Security Groups, EIP)
CI/CD	GitHub Actions (Maven + SSH deploy)
Monitoreo	Prometheus + Grafana (Micrometer + Actuator)
FinOps	Tags de costo + EventBridge Scheduler

2. Contenedores — Docker

2.1 Dockerfile multi-stage

El `Dockerfile` utiliza una construcción **multi-stage** para optimizar el tamaño de la imagen final:

- **Stage 1 — Builder:** Imagen `maven:3.9-eclipse-temurin-17-alpine`. Copia primero el `pom.xml` para cachear dependencias (`mvn dependency:go-offline`), luego copia el código fuente y compila con `mvn clean package -DskipTests`.
- **Stage 2 — Runtime:** Imagen `eclipse-temurin:17-jre-alpine` (solo JRE, ~150 MB vs ~400 MB del JDK). Copia el JAR generado, declara el volumen `/data` para SQLite, expone el puerto `8080` y ejecuta la aplicación.

```
# BUILD STAGE
FROM maven:3.9-eclipse-temurin-17-alpine AS builder
WORKDIR /app
COPY pom.xml .
RUN mvn dependency:go-offline -B
COPY /src ./src
RUN mvn clean package -DskipTests

# RUNTIME STAGE
FROM eclipse-temurin:17-jre-alpine
WORKDIR /app
COPY --from=builder /app/target/*.jar app.jar
VOLUME /data
EXPOSE 8080
CMD ["sh", "-c", "mkdir -p /data && java -jar app.jar"]
```

Optimizaciones aplicadas:

- **Cache de capas:** `COPY pom.xml` y `RUN mvn dependency:go-offline` están antes de copiar el código fuente. Así, si solo cambia el código, Maven reusa las dependencias cacheadas de Docker.
- **Imagen final liviana:** Usa JRE en Alpine, eliminando herramientas de compilación innecesarias en producción.
- **Volumen externo:** Los datos de SQLite persisten fuera del contenedor mediante `VOLUME /data`.

2.2 Docker Compose — Entorno local

El archivo `docker-compose.yml` define el servicio `backend` que construye la imagen desde el `Dockerfile` y monta un volumen `sqlite_data` en `/data` para persistencia de la base de datos.

```
services:
  backend:
    build:
      context: ./back
      dockerfile: Dockerfile
    container_name: todo-api
    ports:
      - "8080:8080"
    volumes:
      - sqlite_data:/data
```

```
volumes:
  sqlite_data:
```

2.3 Docker Compose — Monitoreo

El archivo `monitoring/docker-compose.monitoring.yml` agrega Prometheus y Grafana al stack. Se ejecuta combinado con el principal:

```
docker compose -f docker-compose.yml -f monitoring/docker-compose.monitoring.yml up
```

Prometheus (`prom/prometheus:v2.53.4`): Escucha en el puerto `9090` , monta la configuración `prometheus.yml` y un volumen para datos time-series. Scrapea el endpoint `/actuator/prometheus` del backend cada 15 segundos.

Grafana (`grafana/grafana:11.3.0`): Escucha en el puerto `3000` , monta la configuración de provisioning para auto-conectar el datasource de Prometheus y cargar el dashboard predefinido.

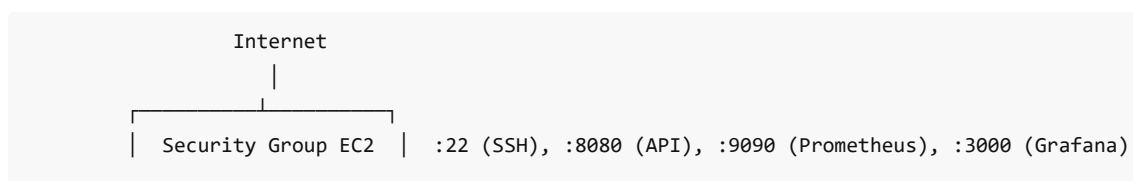
```
#16 exporting to image
#16 exporting layers
#16 exporting layers 2.8s done
#16 exporting manifest sha256:3c597477efe6f9aa5e04d3a2fae66c56ebf21c62f182e89fd2237ea6556898ae 0.0s done
#16 exporting config sha256:049fa9a39abd27331ce3178b9d01a64760f407eed9e2baa3d47fa655a0d7dc 0.0s done
#16 exporting attestation manifest sha256:3761f6338eae0d09dcabb16b80c83a7b04bce6bd4b35893200fd36a3d6e9f8ab 0.0s done
#16 exporting manifest list sha256:ccf07573d790f5232b6af8aff4078e49e0349764c1f33e0aa027b66ba4249abb 0.0s done
#16 naming to docker.io/library/entregafinalcoderhousedevops-backend:latest done
#16 unpacking to docker.io/library/entregafinalcoderhousedevops-backend:latest
#16 unpacking to docker.io/library/entregafinalcoderhousedevops-backend:latest 0.4s done
#16 DONE 3.3s
[+] up 32/32
✓ Image grafana/grafana:11.3.0 Pulled 25.3s
✓ Image prom/prometheus:v2.53.4 Pulled 21.3s
✓ Image entregafinalcoderhousedevops-backend Built 74.9s
✓ Network entregafinalcoderhousedevops_default Created 0.1s
✓ Volume entregafinalcoderhousedevops_grafana_data Created 0.0s
✓ Volume entregafinalcoderhousedevops_sqlite_data Created 0.0s
✓ Volume entregafinalcoderhousedevops_prometheus_data Created 0.0s
✓ Container todo-api Started 2.4s
✓ Container prometheus Started 2.4s
✓ Container grafana Started 2.4s
odnr2@DESKTOP-8TJ7R0P MINGW64 ~/Documents/GitHub/EntregaFinalCoderHouseDevOps (main)
$ |
```

```
odnr2@DESKTOP-8TJ7R0P MINGW64 ~/Documents/GitHub/EntregaFinalCoderHouseDevOps (main)
$ curl http://localhost:8080/api/health
{"service":"EntregaFinal Backend","status":"OK"}
odnr2@DESKTOP-8TJ7R0P MINGW64 ~/Documents/GitHub/EntregaFinalCoderHouseDevOps (main)
$ curl -X POST http://localhost:8080/api/tareas \
-H "Content-Type: application/json" \
-d '{"nombre":"Estudiar DevOps","descripcion":"Preparar entrega final","completada":false}'
{"id":1,"nombre":"Estudiar DevOps","descripcion":"Preparar entrega final","completada":false}
odnr2@DESKTOP-8TJ7R0P MINGW64 ~/Documents/GitHub/EntregaFinalCoderHouseDevOps (main)
$ curl http://localhost:8080/api/tareas
[{"id":1,"nombre":"Estudiar DevOps","descripcion":"Preparar entrega final","completada":false}]
odnr2@DESKTOP-8TJ7R0P MINGW64 ~/Documents/GitHub/EntregaFinalCoderHouseDevOps (main)
$ |
```

3. Infraestructura como Código — Terraform

3.1 Arquitectura

Terraform provisiona una instancia EC2 en AWS con Docker Engine, clona el repositorio de la aplicación, y levanta la API junto con el stack de monitoreo mediante Docker Compose y systemd.



(EC2 t3.micro)	
Docker Compose	← systemd (Restart=always)
SQLite (/data)	

3.2 Módulos

Módulo	Recursos	Función
security	aws_security_group	Puertos SSH (22), API (8080), Prometheus (9090), Grafana (3000)
compute	tls_private_key, aws_key_pair, aws_instance, aws_eip, local_file	Instancia EC2 con EIP, clave SSH, user_data con Docker + systemd
scheduler	aws_iam_role, aws_iam_role_policy, aws_scheduler_schedule	Apagado/encendido automático en horario laboral

3.3 Variables principales

Variable	Default	Descripción
aws_region	us-east-1	Región de AWS
project_name	todo-api	Prefijo para nombrar recursos
app_repo_url	(obligatorio)	URL del repositorio para clonar en la EC2
instance_type	t3.micro	Tipo de instancia (free tier)
environment	dev	Entorno (dev/prod)
owner	(obligatorio)	Responsable (para cost allocation)
monitoring_enabled	true	Desplegar Prometheus + Grafana

3.4 User Data (bootstrap de la EC2)

El script `user_data.sh.tpl` se inyecta en la EC2 al arrancar y realiza:

1. Instala `git` y `docker` via `dnf`
2. Clona (o actualiza) el repositorio en `/opt/app`
3. Construye las imágenes con `docker compose -f docker-compose.yml -f monitoring/docker-compose.monitoring.yml build`
4. Crea un servicio `systemd` que ejecuta Docker Compose con ambos archivos
5. Habilita e inicia el servicio (la app se levanta automáticamente al bootear la instancia)

```
ExecStart=/usr/bin/docker compose -f docker-compose.yml -f monitoring/docker-
compose.monitoring.yml up
```

3.5 Outputs

Tras `terraform apply`, se obtienen:

- `api_url` → `http://<IP>:8080`
- `ec2_public_ip` → IP pública
- `ssh_command` → `ssh -i clave.pem ec2-user@<IP>`
- `monitoring_urls` → URLs de Prometheus (`:9090`) y Grafana (`:3000`)
- `shutdown_schedule` → Horario de auto-apagado

4. CI/CD — GitHub Actions

4.1 Pipeline

El pipeline se define en `.github/workflows/ci.yml` y se ejecuta automáticamente al hacer push a `main` o `master`.

Jobs:

Job	Descripción
build	Compila el proyecto con Maven y sube el JAR como artifact
deploy	Se conecta por SSH a la EC2, hace <code>git pull</code> y reinicia <code>systemd</code> (levanta API + monitoreo)

Flujo del deploy:

1. `actions/checkout@v4` — clona el repositorio
2. `actions/setup-java@v4` — configura JDK 17 con caché de Maven
3. `mvn clean package -DskipTests` — compila y empaqueta en `back/target/*.jar`
4. `actions/upload-artifact@v4` — sube el JAR como artifact
5. `appleboy/ssh-action@v1.2.0` — se conecta por SSH a la EC2 y ejecuta:

```
cd /opt/app
git pull origin main
sudo systemctl restart app
```

4.2 Secrets requeridos

Configurar en GitHub → Settings → Secrets and variables → Actions:

Secret	Descripción
EC2_HOST	IP pública de la EC2
EC2_SSH_KEY	Contenido del archivo <code>clave.pem</code> generado por Terraform

5. Orquestación — Kubernetes

5.1 Manifiestos

Archivo	Recurso	Descripción
<code>01-namespace.yml</code>	Namespace	Aísla la aplicación en <code>todo-app</code>

03-backend-deployment.yml	Deployment	2 réplicas, probes de health, recursos limitados, emptyDir para SQLite
04-backend-service.yml	Service	Expone el puerto 8080 internamente (ClusterIP)
05-backend-hpa.yml	HorizontalPodAutoscaler	Escala de 2 a 6 réplicas según CPU (50%) y memoria (70%)
06-ingress.yml	Ingress	NGINX Ingress Controller, rutea / al backend
07-servicemonitor.yml	ServiceMonitor	Prometheus descubre automáticamente los pods del backend
08-grafana-dashboard-configmap.yml	ConfigMap	Dashboard de Grafana precargado

5.2 Deployment

El Deployment `backend-api` incluye:

- **Readiness probe:** `GET /api/health` cada 5 segundos (delay inicial 15s). El pod solo recibe tráfico cuando responde OK.
- **Liveness probe:** `GET /api/health` cada 10 segundos (delay inicial 30s). Si falla, Kubernetes reinicia el pod.
- **Recursos:** Requests 256Mi/200m, Limits 512Mi/400m.
- **Persistencia:** Volumen `emptyDir` en `/data` para SQLite (efímero, adecuado para desarrollo/pruebas).

5.3 Auto-escalado (HPA)

El HPA escala el número de réplicas del deployment entre 2 y 6 basándose en:

- **CPU:** Utilización promedio del 50%
- **Memoria:** Utilización promedio del 70%

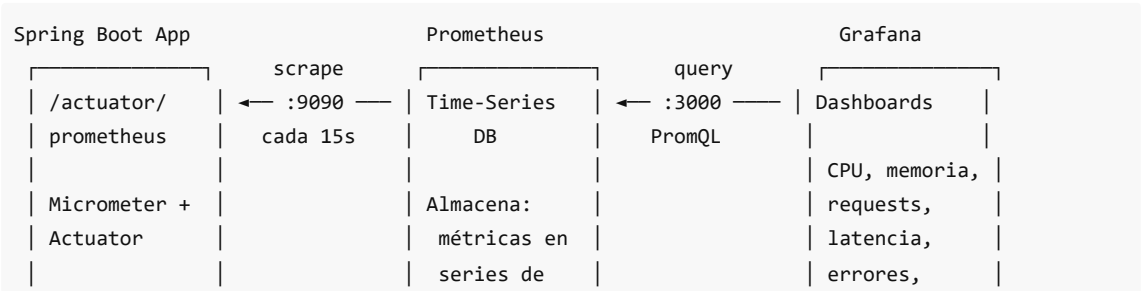
Requiere `metrics-server` instalado en el clúster.

5.4 ServiceMonitor

El `ServiceMonitor` le indica a Prometheus (instalado vía `kube-prometheus-stack`) que descubra y scrapee automáticamente los pods con label `app: backend-api` en el namespace `todo-app`, en el puerto `8080` y ruta `/actuator/prometheus`.

6. Monitoreo — Prometheus y Grafana

6.1 Arquitectura





6.2 Spring Boot — Exposición de métricas

Se agregaron dos dependencias al `pom.xml` :

- `spring-boot-starter-actuator` — endpoints de monitoreo (`/actuator/health` , `/actuator/info`)
- `micrometer-registry-prometheus` — exporta las métricas en formato Prometheus (`/actuator/prometheus`)

```
management.endpoints.web.exposure.include=health,info,prometheus
management.endpoint.prometheus.enabled=true
management.metrics.tags.application=${spring.application.name:todo-api}
```

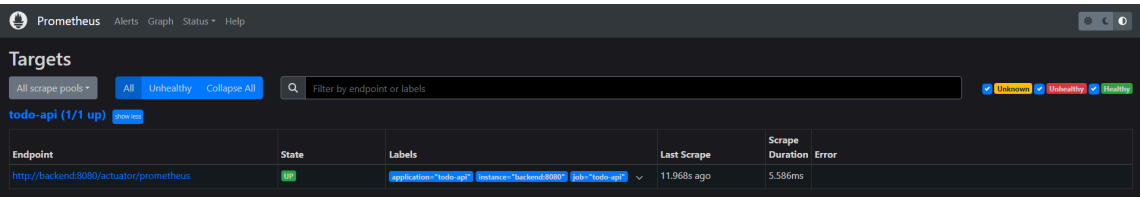
Spring Boot expone automáticamente métricas de:

- JVM (memoria heap/non-heap, threads, garbage collector)
- HTTP (requests por endpoint/status/método, latencia en buckets)
- Sistema (CPU de proceso y del sistema)
- Logs (eventos por nivel)

6.3 Prometheus — Configuración de scrapeo

```
global:
  scrape_interval: 15s

scrape_configs:
  - job_name: "todo-api"
    metrics_path: /actuator/prometheus
    static_configs:
      - targets: ["backend:8080"]
```

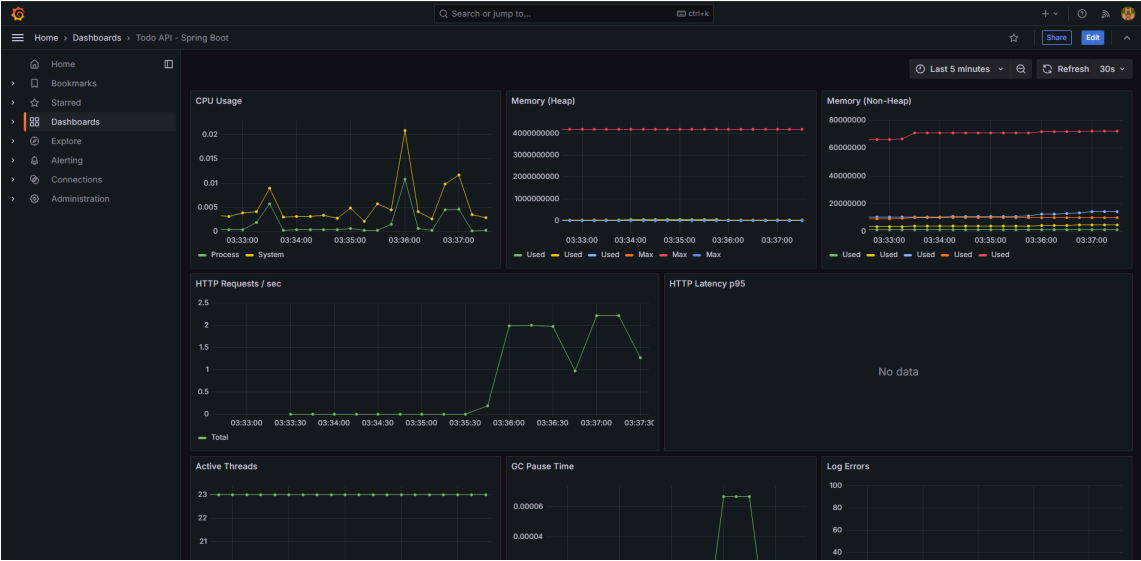


6.4 Dashboard de Grafana

El dashboard precargado incluye 8 paneles:

Panel	Métrica PromQL	Propósito
CPU Usage	<code>process_cpu_usage</code>	Carga de CPU del proceso
Memory (Heap)	<code>jvm_memory_used_bytes{area="heap"}</code>	Memoria heap usada vs máxima
Memory (Non-Heap)	<code>jvm_memory_used_bytes{area="nonheap"}</code>	Metaspace y otros

HTTP Requests/s	rate(http_server_requests_seconds_count[1m])	Tráfico por segundo
HTTP Latency p95	histogram_quantile(0.95, ...)	Percentil 95 de latencia
Active Threads	jvm_threads_live_threads	Threads activos
GC Pause Time	rate(jvm_gc_pause_seconds_sum[1m])	Tiempo de garbage collector
Log Errors	rate(logback_events_total{level="error"}[1m])	Errores por segundo



6.5 Despliegue en los 3 entornos

Entorno	Comando / Método
Local	<code>docker compose -f docker-compose.yml -f monitoring/docker-compose.monitoring.yml up</code>
Kubernetes	<code>helm install prometheus-stack prometheus-community/kube-prometheus-stack + kubectl apply -f k8s/07-servicemonitor.yml</code>
AWS (EC2)	Automático via <code>user_data.sh.tp1</code> — systemd ejecuta ambos compose files

7. FinOps — Gestión de Costos en AWS

Se implementó el ciclo FinOps en 3 fases:

7.1 Inform — Tags de cost allocation

Todos los recursos de AWS creados por Terraform heredan automáticamente los siguientes tags vía `default_tags` :

Tag	Valor	Propósito
Environment	dev	Identifica el entorno
Project	todo-api	Agrupar costos por proyecto

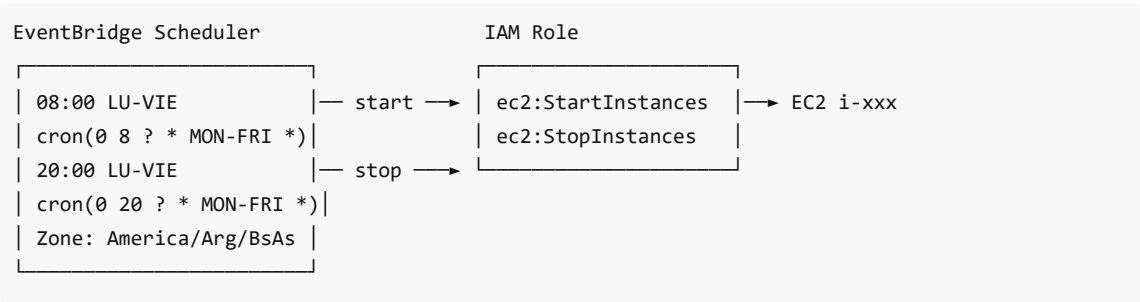
Owner	(configurable)	Responsable del recurso
ManagedBy	Terraform	Origen del recurso

Además, cada recurso tiene un tag `Name` específico (ej: `todo-api-ec2` , `todo-api-ec2-sg` , `todo-api-eip`).

Esto permite filtrar costos en **AWS Cost Explorer** y asignar gastos a proyectos específicos.

7.2 Optimize — Auto-shutdown con EventBridge Scheduler

Arquitectura:



- Solo se activa en entorno `dev`
- Apaga a las 20:00 ART, enciende a las 08:00 ART, solo días hábiles
- Al reiniciar la EC2, systemd levanta automáticamente la app y el monitoreo

Ahorro estimado:

Métrica	24/7	Auto-shutdown
Horas/mes	730	~260
Costo EC2 t3.micro	~\$8.47/mes	~\$3.01/mes
Ahorro	—	~64%

7.3 Operate — Budgets y documentación

Se documentó en `Documentacion/FINOPS.md` :

- Desglose completo de costos mensuales
- Configuración recomendada de AWS Budgets (alerta a \$12/\$15)
- Estrategias de ahorro adicionales (Reserved Instances, Spot, Graviton)

8. Estructura del Proyecto

```
EntregaFinalCoderHouseDevOps/
|
├── back/                                # API Spring Boot
|   ├── Dockerfile                      # Multi-stage (Maven → JRE)
|   ├── pom.xml                         # Dependencias (Web, JPA, SQLite, Actuator,
Micrometer)
|   ├── mvnw / mvnw.cmd                 # Maven Wrapper
|   └── src/main/java/com/entregafinal/demo/
```

```

├── DemoApplication.java
├── controller/                # HealthController, TareaController
├── model/Tarea.java           # Entidad JPA
├── service/TareaService.java
└── repository/TareaRepository.java

├── monitoring/                # Stack de monitoreo
├── ├── prometheus.yml          # Config de scrapeo
├── ├── docker-compose.monitoring.yml # Prometheus :9090 + Grafana :3000
├── └── grafana/provisioning/    # Datasource + dashboard auto

├── k8s/                       # Manifiestos Kubernetes
├── ├── 01-namespace.yml        # Namespace todo-app
├── ├── 03-backend-deployment.yml # Deployment con probes y recursos
├── ├── 04-backend-service.yml   # Service (ClusterIP :8080)
├── ├── 05-backend-hpa.yml       # HPA (CPU 50%, RAM 70%)
├── ├── 06-ingress.yml           # Ingress NGINX
├── ├── 07-servicemonitor.yml    # Prometheus auto-descubrimiento
├── └── 08-grafana-dashboard-configmap.yml

├── infraestructura/           # Terraform – AWS
├── ├── versions.tf             # Providers + default_tags
├── ├── variables.tf            # Variables de entrada
├── ├── main.tf                 # Data sources + módulos
├── ├── outputs.tf              # Outputs (URLs, IP, SSH, schedule)
├── ├── scheduler.tf            # EventBridge auto-shutdown
├── ├── terraform.tfvars.example
├── └── modules/
├── │   ├── security/           # Security Group
├── │   └── compute/            # EC2 + EIP + key pair + user_data

├── .github/workflows/ci.yml    # Pipeline CI/CD
├── docker-compose.yml          # Entorno local (API + SQLite)
├── .gitignore
├──
└── Documentacion/
    ├── README.md              # Documentación principal
    ├── FINOPS.md              # Gestión de costos AWS
    └── informe.md              # Este informe

```

9. Conclusión

Se implementó un pipeline DevOps completo que cubre:

1. **Desarrollo:** API REST funcional con Spring Boot y SQLite
2. **Contenerización:** Docker multi-stage optimizado con Docker Compose
3. **Infraestructura como código:** Terraform para provisionar EC2 en AWS con todas las configuraciones necesarias
4. **CI/CD:** Pipeline automatizado que compila, empaqueta y despliega en AWS ante cada push
5. **Orquestación:** Kubernetes con deployment, service, HPA, ingress y health checks
6. **Monitoreo:** Prometheus + Grafana con métricas de JVM, HTTP, CPU y dashboards precargados

7. **FinOps:** Tags de cost allocation y auto-shutdown para optimizar gastos en la nube

El proyecto está diseñado para que cualquier desarrollador pueda clonar el repositorio, ejecutar la aplicación localmente con `docker compose` , desplegarla en Kubernetes con `kubect1 apply -f k8s/` , o provisionar infraestructura completa en AWS con `terraform apply` , todo sin asistencia adicional.